

2026년 오픈소스 개발자대회 결과보고서

항 목	내 용	항 목	내 용
팀 명	[팀명 입력 필요 - 참가 접수 정보와 대조]	팀 인 원 (팀장 포함)	[N]명
참가부문	일반	과제유형	자유과제

□ 결과보고서

프로젝트 개요	
프로젝트명	SongRyeon Core v1 (접수 작품명과 최종 대조 필요)
프로젝트 등록 URL	https://github.com/Junghoo-developer/SongRyeon/tree/refoundation/songryeon-v1
시연영상	[3분 이하 YouTube 시연 영상 URL 입력 필요]
프로젝트 소개	코드가 확인한 실행 사실(A)과 LLM이 만든 판단(R)을 원자 기록으로 분리하고, 증거 수집·충분성 검사·답변·검열을 네 노드로 나눈 로컬 오픈 웨이트 에이전트 런타임.
프로젝트 세부 내용	
개발배경 및 목적	문제 LLM 에이전트는 실제로 실행한 도구·읽은 파일과 모델이 추론한 내용을 같은 문맥에서 섞기 쉽다. 이때 모델이 실행 동선까지 잘못 말하면 사용자는 답변의 근거를 추적하기 어렵다. 목표 세상 모든 문장의 진위를 판정하는 대신, 코드가 그대로 확인할 수 있는 실행 사실과 모델의 해석을 저장·공개·검열 단계에서 분리한다. 모델의 모든 서술은 R로, 언제 어느 노드가 무엇을 실행해 어떤 원문을 얻었는지는 A로 남긴다.
개발환경	언어·저장 Python 3.10 이상, UTF-8 JSONL 원본 기억, SQLite 지식 색인. 필수 Python 런타임 의존성 없이 표준 라이브러리로 구성했다. 모델 실행 Ollama 0.32.5의 로컬 HTTP 인터페이스. 제출 프로파일은 gemma4:26b(Q4_K_M, 요청 컨텍스트 16,384, temperature 0, seed

	42), qwen3:14b는 비교 기준선이다. 검증 환경(2026-07-30 관찰값) Windows 11 Pro 64-bit, Intel Core i9-14900K, NVIDIA GeForce RTX 5080 16,303 MiB, 물리 메모리 63.69 GiB, Python 3.10.11. 이는 최소 사양이 아니라 검증 호스트 기록이다.
시스템 구성 및 아키텍처	요청 → Node1 → Node2 → Node3 → Node4 → 최종 답변 Node1만 프로젝트 경계 안의 읽기 전용 Python 파일 도구를 사용한다. 도구 원문은 감사 로그에 보존되고, Node1이 full-excerpt-결정론적 chunk-omit 중 공개 방식을 고르면 코드는 원문에서 선택 구간을 그대로 복사한다. Node2는 요청에 필요한 A가 충분한지 검사하고, Node3는 사용자 답변을 작성하며, Node4는 답변의 현재 코드 주장을 공개된 tool_result_content A와 대조한다. Node2·Node4는 각각 반려 3회 상한을 가지며 상한 초과는 permit이 아니라 검증 미완료 상태로 표시된다. 원본 기억 7필드: information, information_class, code_verifiable, information_type, turn_id, information_id, created_at. 에이전트 시야는 숨김 원문과 UUID·시각을 제외한 원자에 원본 줄 기반 memory_index를 붙인다. 턴 시작 시 잡은 가장 오래된 공개 원자를 턴 종료까지 기준점으로 고정한다.
프로젝트 주요기능	핵심 기능 1) append-only JSONL에 A/R 원자와 노드·도구 동선을 기록. 2) 모델이 보존 원문을 재작성하지 못하도록 코드가 정확한 구간을 복사. 3) 절대경로·상위경로 탈출·심볼릭 링크 이탈·비 Python 파일·크기 상한 초과를 파일 도구에서 거부. 4) Node2 증거 충분성 검사와 Node4 답변 근거 검열을 분리. 개발 과정 저장 레코드와 테스트 격리부터 시작해 에이전트 시야, 도구 원문 보존, 전부 omit 복구, 유한 반려 루프, 로컬 모델 JSON 호출 순으로 작은 모듈과 결정론적 테스트를 추가했다. 실제 데모에서 과거 요청 반복, 근거 없는 부재 주장, 원문 전부 omit 문제를 확인하고 프롬프트·라우팅·상태 표시를 보완했다. 구동·시험 설치 후 python -m pytest -q, 이어서 python -m demo --memory ./tmp/demo-memory.jsonl "질문"으로 실행한다. 실제 memory/memory.jsonl 대신 격리 로그를 쓰면 원본 기억을 건드리지

	않고 Node1~4 이벤트와 최종 답변을 재현할 수 있다. 2026-07-30 작업 트리에서 230 passed, 1 skipped를 확인했고, wheel 새 가상환경 설치 뒤 10개 fixture 재생도 통과했다. 평가 경계 python -m evals의 10개 합성 fixture는 평가기 계약과 해시·완전행렬 검증용이며 실제 모델 성능 수치가 아니다. 동일 qwen3:14b로 10 case×3구조 원시 파일럿 30건과 실패 1건을 모두 보존했지만 사람 정규화 전이므로 성능 수치로 인용하지 않는다. 라이브 비교는 동일 조건의 원문을 보존하고 review_status=reviewed인 결과만 인용한다.
기대효과 및 활용분야	기대 효과 코드 조사·리뷰형 에이전트에서 '실제로 무엇을 읽고 실행했는가'와 '모델이 무엇을 판단했는가'를 다시 추적할 수 있다. 원본 감사 로그와 제한된 모델 시야를 분리해 출처 보존과 문맥 비용을 함께 다루는 기반을 제공한다. 활용·확장 읽기 전용 코드 조사, 내부 문서 검토, 실행 동선 감사에 적용할 수 있다. 이후 지식 DB·과거 기억 검색, 원본 불변 파생 압축, 추가 읽기 도구를 같은 A/R 계약 아래 확장할 수 있다.
기타	차별성 더 큰 프롬프트 하나에 신뢰를 맡기지 않는다. 모델 출력은 끝까지 R로 유지하고, 도구 원문·카운터·라우팅은 코드가 A로 기록한다. 증거를 고르는 Node1, 충분성을 보는 Node2, 답하는 Node3, 근거를 검열하는 Node4를 분리하며 반려 상한 초과도 성공으로 위장하지 않는다. 현재 한계 데모 도구는 .py 목록·열람에 집중한다. 지식 색인은 아직 Node1 검색 도구와 연결되지 않았고 과거 기억 검색·압축도 없다. 모델 판단은 R이므로 오류가 남을 수 있고, 반려 상한 초과 답변은 반환되더라도 검증 미완료다. 로드맵 동일 모델 3개 구조 변형의 공개 비교평가 → 지식·기억 검색 도구 → 원본 보존형 파생 시야 압축 → 다중 writer 잠금과 장기 실행 관측 순으로 확장한다. 개발 소감 [참가자가 직접 작성 필요: 문제 정의, A/R 분리를 구현하며 겪은 시행착오, 실제 데모와 테스트에서 배운 점을 본인의 경험으로 3~5문장 기재]

붙임1

SBOM[소프트웨어 자재명세서]

번호	라이브러리명	버전	라이선스	공식 저장소 URL(GitHub 등)	사용 목적 및 주요 기능
1	Python / 표준 라이브러리	3.10.11	PSF-2.0	https://www.python.org/	애플리케이션 실행, JSON·SQLite·HTTP·경로 처리
2	setuptools	65.5.0	MIT	https://github.com/pypa/setuptools	PEP 517 빌드 backend
3	pytest	9.0.3	MIT	https://github.com/pytest-dev/pytest	선택적 개발·회귀 테스트
4	Ollama	0.32.5	MIT	https://github.com/ollama/ollama	로컬·자체 호스팅 모델 HTTP 실행
5	Gemma 4 26B A4B	5571076f3d70 / Q4_K_M	Apache-2.0	https://ollama.com/library/gemma4:26b	대회 제출 프로파일의 로컬 생성 모델
6	Qwen3-14B	bdbd181c33f2 / Q4_K_M	Apache-2.0	https://huggingface.co/Qwen/Qwen3-14B	로컬 비교평가 기준선(자동 fallback 아님)

※ 필요 시, 행을 추가하여 기재해 주시기 바랍니다.

붙임2

AI 모델 활용 및 라이선스 기술 명세서

[작성 안내]

- 본 서식은 프로젝트에 탑재·적용된 AI 모델의 오픈소스 요건 및 라이선스 준수 여부를 검증하기 위한 기술 명세서입니다.
- 참가팀은 본인의 AI 개발 유형(유형1, 2, 3)을 먼저 선택한 후 해당하는 항목의 명세를 정확히 작성하여 제출해 주시기 바랍니다.

1. AI 모델 활용 유형 (해당하는 항목에 ☒ 표시)

☒ 유형 1: 외부 공개 모델을 추가 학습 없이 프로젝트에 연동·구동

☐ 유형 2: 외부 모델 파인튜닝

☐ 유형 3: 자체 개발 모델

개발 과정의 상용 AI 코딩 보조 사용은 아래 4번 항목에 별도 공개하며, 실행 모델의 유형 선택과 구분한다.

2. 기반(베이스) 모델 정보 (유형 1, 유형 2 작성 필수 / 유형 3은 '해당 없음'으로 기재)

기반 모델 명 및 개 발사	Gemma 4 26B A4B (Google DeepMind), Ollama tag gemma4:26b, 25.8B parameters, Q4_K_M; digest 5571076f3d70050487b26b341705799e0ab29b808164f90d20d4cf84f699d251	기반 모델 라이선스	Apache License 2.0 (https://ollama.com/library/gemma4:26b)
----------------------	---	---------------	---

3. 데이터셋 정보 및 가중치 배포 명세 (유형 2, 유형 3 작성 필수)

학습 데이 터셋 정보 (출처 및 규모)	해당 없음 - 유형 1로 추가 학습·파인튜닝을 수행하지 않았으며 학습 데이터셋이나 새 가중치를 생성하지 않음.
데이터 정 제/ 가공 방법 요약	해당 없음 - 유형 1로 추가 학습·파인튜닝을 수행하지 않았으며 학습 데이터셋이나 새 가중치를 생성하지 않음.
새로 생성	해당 없음 - 유형 1로 추가 학습·파인튜닝을 수행하지 않았으며 학습 데이터셋이나 새 가중치를 생성하지 않음.

된 가중치 공개 저장 소 URL			
가중치 파 일 정보 및 배포방 식	해당 없음 - 유형 1로 추가 학습·파인튜닝을 수행하지 않았으며 학습 데이터셋이나 새 가중치를 생성하지 않음.		
4. 소스코드 라이선스 및 개발 환경 정보 (모든 유형 필수 작성)			
직접 작성 한 코드의 오픈소스 라이선스	MIT License (저장소 LICENSE 포함)	학습/추론 소스코드 공개 저장 소 URL	https://github.com/Junghoo-developer/SongRyeon/tree/refoundation/songryeon-v1
상용 AI 보조도구 활용 여부 및 범위	OpenAI Codex를 설계 토론, 코드 초안·수정, 테스트 작성·실행, 문서 검토 보조에 사용했다. 참가자가 목표와 구조를 결정하고 실제 데모 출력·프롬프트·테스트 결과를 반복 확인해 제안을 수정·채택했다. AI 보조가 만든 일부 코드·문서 초안은 사람의 검토와 회귀 테스트를 거쳐 출품 소스에 포함되었다. 사용 비율은 임의로 산정하지 않았다.		